

Problem Set on CPU Virtualization

1. Briefly state the main steps that take place during transitioning from user mode to kernel mode via a system call.
2. State the sequence of important steps that take place during a system call clearly mentioning whether it is done by the OS, H/W or user program?
3. During the context switch mechanism, saving/restoring of two types of register context happens. Clearly explain the differences between them.
4. A process executes following line of C code “if (fork()!=0) {fork();} fork();”. How many NEW processes are created?
5. Consider a parent process P that has forked a child process C in the program below.

```
int a = 5;
int fd = open(...) //opening a file
int ret = fork();
if(ret>0)    {
    close(fd); a
    = 6;
    ...
}
else if(ret==0) { printf("a=%d\n", a);
read(fd, something);
}
```

After the new process is forked, suppose that the parent process is scheduled first, before the child process. Once the parent resumes after fork, it closes the file descriptor and changes the value of a variable as shown above. Assume that the child process is scheduled for the first time only after the parent completes these two changes.

- (a) What is the value of the variable a as printed in the child process, when it is scheduled next? Explain.
 - (b) Will the attempt to read from the file descriptor succeed in the child? Explain.
6. Consider a parent process that has forked a child in the code snippet below.

```
int count = 0; ret =
fork(); if(ret == 0)    {
printf("count in child=%d\n", count);
}
else    {
count = 1; }
```

The parent executes the statement "count = 1" before the child executes for the first time. Now, what is the value of count printed by the code above? Assume that the OS implements a regular fork (not a copy-on-write fork).

7. Consider a process that has requested to read some data from the disk and blocks. Subsequently, the data from the disk arrives and the interrupt is serviced. However, the process doesn't start running immediately. What is the state of this process at this stage?
8. Consider a process P1 that forks P2, P2 forks P3, and P3 forks P4. P1 and P2 continue to execute while P3 terminates. Now, when P4 terminates, which process must wait for and reap P4?
9. Consider the following program:

```
main(){
    int fd;
    fd = open("outfile", O_CREAT|O_RDWR); // Line 1
    fork(); // Line 2
    write(fd, "hello", 5);
    exit(0);
}
```

 - a) Assume that all system calls succeed on a uniprocessor system. Also, assume that a system call cannot be interrupted in the middle of its execution. What will be the contents of the "outfile" file, after all processes have successfully exited?
 - b) What will be the contents of "outfile" if fork() was called before open() i.e, Line 2 comes before Line 1 in the program (see comments for identifying Line 1 & 2) above.
10. Consider the following three processes that arrive in a system at the specified times, along with the duration of their CPU bursts. Process P1 arrives at time $t=0$, and has a CPU burst of 10 time units. P2 arrives at $t=2$, and has a CPU burst of 2 units. P3 arrives at $t=3$, and has a CPU burst of 3 units. Assume that the processes execute only once for the duration of their CPU burst, and terminate immediately. Calculate the time of completion of the three processes under each of the following scheduling policies. For each policy, you must state the completion time of all three processes, P1, P2, and P3. Assume there are no other processes in the scheduler's queue. For the preemptive policies, assume that a running process can be immediately preempted as soon as the new process arrives (if the policy should decide to preempt).
 - (a) First Come First Serve
 - (b) Shortest Job First (non-preemptive)
 - (c) Shortest Remaining Time First (preemptive)
 - (d) Round robin (preemptive) with a time slice of (atmost) 5 units per process

11. Consider the following set of processes, with the length of the CPU burst given in milliseconds.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0.

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, non-preemptive priority (a smaller priority number implies a higher priority), and RR (quantum = 1).
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of the scheduling algorithms in part a?
- Which of the algorithms on part a results in the minimum average waiting time (over all processes)?

12. Consider the following set of processes, with the length of the CPU burst and arrival time given in milliseconds.

Process	Burst Time	Arrival time
P1	8	0
P2	4	0.4
P3	1	1

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, Clairvoyant SJF (the algorithm can look into the future and wait for a shorter process that will arrive).
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of the scheduling algorithms in part a?
- Which of the algorithms from part a results in the minimum average waiting time (over all processes)?

13. In the context of Round Robin (RR) scheduling, answer the following:

- What are the major strengths and weaknesses of the RR approach?
- How does reducing the time quanta of RR change its behavior/performance?
- If a random job is picked to run next at each quantum, how would that change its performance (is it better, worse, or quite similar)?

14. Consider a variant of the Round Robin (RR) scheduling algorithm in which the entries in the ready queue are pointers to the PCBs.

- What would be the effect of putting two pointers to the same process in the ready queue?

- b) What would be major advantages and disadvantages of this scheme?
- c) How would you modify the basic RR algorithm to achieve the same effect without the duplicate pointers?

15. In a particular workload given in table below, Jobs A and B are completely CPU-bound processes. Job C requires 1 unit of CPU followed by 4 time units of I/O, in a pattern repeated thrice, followed by 1 unit of CPU. For each job sequence, C stands for CPU and I stands for I/O.

Job ID	Arrival	Execution pattern	Length
A	0	CCCCCCCCCCC	10
B	1	CCCCCCCCCCC	11
C	2	CIIIIIIIIIIC	16

An example job schedule of a FIFO scheduling policy (without preemption) would look like:
 AAAAAAAAAABBBBBBBBBBBBCxxxxCxxxxCxxxxC

where x represents an idle CPU. Note here Job A began at time 0 and finished at time 10.
 Determine the job schedule for each of the following scheduling policies (a) to (c)

(a) Round Robin with a 1-unit time quantum

(b) SJF

(c) STCF

(d) From the given workload, which of the above scheduling policies maximize(s) CPU

utilization as well as most suited for interactive jobs? If there is more than one, explain the primary advantage of using one over the other.

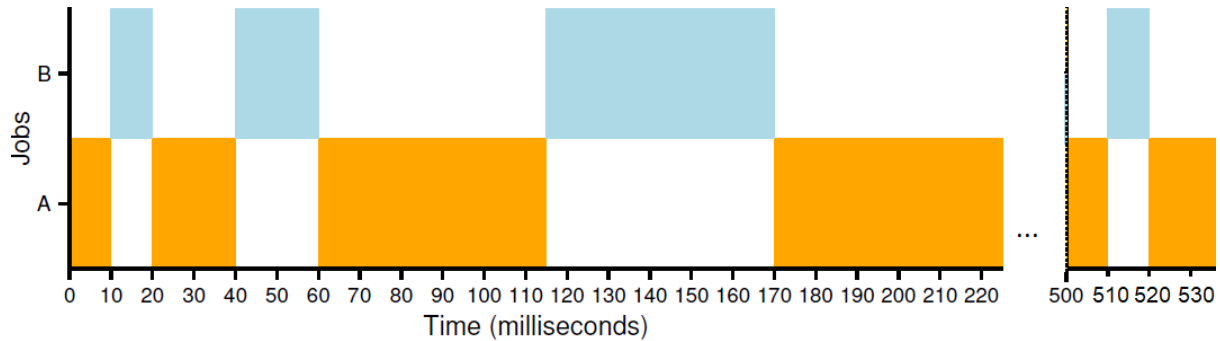
16. Consider a multilevel feedback queue scheduler with three queues, numbered Q0 to Q2 (Q0 being the highest priority queue) and time quantum of 4 ms, 8 ms and 16ms respectively. If a process arrives in a higher priority queue, it will preempt any process in lower queue. Suppose that the following processes arrive for executing at the times indicated (numbers are in milliseconds). Assume no priority boosting.

(a) Draw the scheduling of these processes with respect to time on X axis and different Queue's on the Y axis.

(b) Compute the average waiting and turnaround time.

Process	Arrival Time	Burst Time
P1	0	16
P2	14	6
P3	19	3
P4	20	1
P5	21	14

17. Figure below shows the timeline of how two CPU-bound (no I/O) jobs, A and B run over time on an OS using MLFQ (multi-level feedback queue) scheduler. Assume no other jobs are present in the system. Note that in the timeline after 225 till 500, A and B continue to get scheduled alternately with time quantum of 55 milliseconds and after 500 milliseconds, the entire pattern repeats again, indefinitely (until the jobs are done)



- How many queues do you think there are in this MLFQ scheduler?
- How long is the time slice at the top-most (high priority) and the bottom-most (low priority) queue?
- How often do processes get moved back to the topmost queue?
- Why does the scheduling policy MLFQ move processes to higher priority levels (i.e., the topmost queue) sometimes?